

A Borrow-Aware Compiler from MIRLite to OSEA-IR

ANONYMOUS AUTHOR(S)

MIRLite gives typed Rust-like places an operational semantics. Its compiler lowers each statement to OSEA-IR with explicit reads, retags, writes, and borrow retirement. We present Stacked Borrows, both languages, and the compiler in one format: a grammar, a table of named rules, and the stepwise execution of one five-statement program that allocates a nested tuple, writes a nested field, borrows that field, and writes through the borrow. We then state the forward simulation that relates source and target memory, locals, and per-cell permission stacks, and replay the same program as a simulation. The simulation is mechanized in Lean 4 and covers every MIRLite construct except heap allocation and deallocation.

Additional Key Words and Phrases: borrow semantics, compiler correctness, Lean, Stacked Borrows

1 Introduction

The compiler in this paper connects two views of the same memory operation. MIRLite says *which typed place* a Rust-like statement accesses. OSEA-IR says *which reads, retags, writes, and borrow retirements* implement that access. The proof relates their executions without demanding that their permission stacks be literally equal: the target is allowed to introduce short-lived tags that have no source counterpart. Section 2.4 names them route tags.

We use cells as the unit of layout. Write $|\tau|$ for the number of cells in a layout τ . Naturals and pointers occupy one cell; a tuple occupies the sum of its fields. A pointer value $\text{ptr}(b, o, sz, t)$ denotes address $b + o$ in an allocation with base b , size sz , and provenance tag t . A memory μ maps cell addresses to words, pointers, or undefined values. A permission state Π records the per-cell borrow stacks.

Section 2 formalizes the compilation. It (1) fixes the permission model, per-cell Stacked Borrows, and threads it through every memory operation (Section 2.1); (2) defines MIRLite, the typed source language (Section 2.2); (3) defines OSEA-IR, a register machine whose instructions expose the permission events a MIRLite statement leaves implicit (Section 2.3); (4) presents the compiler between them (Section 2.4); and (5) states the forward simulation that the compiler satisfies (Section 2.5). Section 3 records what is mechanized, and Appendix A extends every table from the constructs shown in the main text to the full executable surface.

READING GUIDE Subsections 2.1 to 2.4 share one layout. *Syntax* is a single figure: grammar, semantic domains, types. *Semantics* is a single table of named rules, read as premises, conclusion, name. The *example* is a stepwise table giving the state after each instruction, and a walkthrough that names the rule each row fires. From Section 2.2 on the example is one program, read once more in Section 2.5 as a simulation.

2 Compiling MIRLite to OSEA-IR with ownership

This section formalizes the compilation from MIRLite to OSEA-IR and the sense in which it preserves ownership. The order is bottom-up. Stacked Borrows comes first (Section 2.1), because both languages are defined over it. MIRLite (Section 2.2) and OSEA-IR (Section 2.3) follow, then the compiler between them (Section 2.4), then its correctness (Section 2.5). Each subsection ends with an example. The example of Section 2.1 is a command sequence on a single cell; from Section 2.2 on it is one MIRLite program, introduced there and carried through the rest of the section.

2.1 Stacked Borrows

$\Pi(a) = [] \xrightarrow{\text{own}(a)} [\text{Own}(t)] \xrightarrow{\text{ref}(\text{mutable}, a, t)} [\text{MutRef}(u), \text{Own}(t)] \xrightarrow{\text{useMut}(a, u)} [\text{MutRef}(u), \text{Own}(t)] \xrightarrow{\text{die}(a, u)} [\text{Own}(t)]$

Fig. 1. A Stacked Borrows command sequence on the one cell a , with the stack after each command.

The Stacked Borrows model of Jung et al. formalizes Rust’s ownership and borrowing rules for safe and unsafe code. It is orthogonal to the rest of a memory model: it is added to a language by threading a permission state through the memory operations. We use that to give MIRLite and OSEA-IR the *same* permission semantics. Formally, both languages are parameterized by a *permission model*, an abstract state Π with partial operations `own`, `read`, `useMut`, `ref`, and `die`. Each returns a new state or fails; a failure is undefined behavior and aborts the execution. The correctness result instantiates the model with the per-cell Stacked Borrows of this subsection. The full executable language uses four further operations, for deallocation, exposed provenance, and protector frames; they are in Appendix A.2.

A permission state is $\Pi = (\text{stacks}, \text{NextTag}, \text{frames}, \text{exposed})$. The component `stacks` is a partial map from cell addresses to *borrow stacks*; `NextTag` is the next fresh tag; `frames` and `exposed` are the protector frames and the list of exposed tags, which only the full language uses (Appendix A.2). We write $\Pi(a)$ for the stack at a and leave the other three components implicit when a rule does not change them. A borrow stack is a list of *items*, topmost first,

$$\iota ::= \text{Own}(t) \mid \text{MutRef}(t) \mid \text{Ref}(t) \mid \text{RawPtr}(m', t) \mid \text{Disabled}(t),$$

where t is the item’s tag and m' a mutability flag. The top of the stack is the most recently derived permission. A new item is derived from an existing one by a *retag*, and the *retag kind*

$$k ::= \text{shared} \mid \text{mutable} \mid \text{raw-const} \mid \text{raw-mut} \mid \text{two-phase}$$

says which item it pushes: a shared reference `Ref`, a mutable reference `MutRef`, a read-only or mutable raw pointer `RawPtr(false, ·)` or `RawPtr(true, ·)`, or a reserved mutable borrow. The kinds `shared` and `mutable` are Rust’s `&T` and `&mut T`, and `raw-const` and `raw-mut` are its raw pointers `*const T` and `*mut T`. A `raw-const` retag acts on the stack exactly as a `shared` one does, but pushes a differently named item; the name matters because related stacks must agree item by item (Definition 2.5). A retag also takes a protector flag c and an interior-mutability mask m , a list of booleans with one entry per cell of the retagged range. Both are inert in the main text, where $c = \text{false}$ and $m = []$, and so is `two-phase`; all three are defined in Appendix A.2. Table 1 gives the rules. A rule $\langle \text{op}, \Pi \rangle \Downarrow_a \Pi'$ acts on one cell a ; “ u fresh” means $u = \Pi.\text{NextTag}$, and the conclusion increments the counter. Fig. 1 runs four of them: **SB-OWN** pushes the owning item of a new allocation, **SB-REF-MUT** derives a mutable borrow with a fresh tag u from its parent t , **SB-USE-MUT** validates a write through u , and **SB-DIE** pops u ’s item, which is how a borrow is retired. After the four commands the stack is what it was after the first; only the tag counter remembers that u existed. That observation is what makes a route tag unobservable at the next statement boundary, and it is why the simulation of Section 2.5 compares tag counters by an inequality.

Three points of the model matter later. A read through t does not remove the mutable borrows above t : **SB-READ** *disables* them in place, because removing them would merge the raw-pointer groups on either side. A `shared` or `raw-constant` retag is a read through its parent followed by a push (**SB-REF-SHARED**). A `raw-mutable` retag performs no access and inserts its item directly above its parent (**SB-REF-RAW**), so sibling raw pointers share one group instead of invalidating each other. And every operation of the interface ranges over n cells from a (**SB-RANGE**): $\text{own}(\Pi, a, n) = (\Pi', t)$, $\text{read}(\Pi, a, n, t) = \Pi'$, $\text{useMut}(\Pi, a, n, t) = \Pi'$, $\text{ref}(\Pi, a, n, t, k, c, m) = (\Pi', u)$, and $\text{die}(\Pi, a, n, t) = \Pi'$. The per-cell granularity is what makes the width of a borrow observable, and hence what the compiler must get right in Section 2.4.

Table 1. Stacked Borrows rules at one cell a . x and y range over lists of items, x being the part of the stack above the granting item; $\text{tag}(i)$ is the tag an item carries; $\text{dis}(x)$ replaces each $\text{MutRef}(u)$ in x by $\text{Disabled}(u)$. In **SB-REF-SHARED**, i' is the item that a retag of kind k pushes. No rule applies when a removed or disabled item is protected (Appendix A.2).

Premises	Conclusion	Rule Name
$\Pi(a) \in \{\perp, []\}, \quad t \text{ fresh}$	$\langle \text{own}(a), \Pi \rangle \Downarrow_a (\Pi[a \mapsto [\text{Own}(t)]], t)$	SB-OWN
$\Pi(a) = x \# (i :: y), \quad \text{tag}(i) = t,$ $i \neq \text{Disabled}(t)$	$\langle \text{read}(a, t), \Pi \rangle \Downarrow_a \Pi[a \mapsto \text{dis}(x) \# (i :: y)]$	SB-READ
$\Pi(a) = x \# (i :: y), \quad \text{tag}(i) = t,$ $i \in \{\text{Own}(t), \text{MutRef}(t)\}$	$\langle \text{useMut}(a, t), \Pi \rangle \Downarrow_a \Pi[a \mapsto i :: y]$	SB-USE-MUT
$\langle \text{useMut}(a, t), \Pi \rangle \Downarrow_a \Pi_1, \quad u \text{ fresh}$	$\langle \text{ref}(\text{mutable}, a, t), \Pi \rangle \Downarrow_a$ $(\Pi_1[a \mapsto \text{MutRef}(u) :: \Pi_1(a)], u)$	SB-REF-MUT
$\langle \text{read}(a, t), \Pi \rangle \Downarrow_a \Pi_1, \quad u \text{ fresh},$ $(k, i') \in \{\text{shared}, \text{Ref}(u)\},$ $(\text{raw-const}, \text{RawPtr}(\text{false}, u))\}$	$\langle \text{ref}(k, a, t), \Pi \rangle \Downarrow_a$ $(\Pi_1[a \mapsto i' :: \Pi_1(a)], u)$	SB-REF-SHARED
$\Pi(a) = x \# (i :: y), \quad \text{tag}(i) = t,$ $i \neq \text{Disabled}(t), \quad u \text{ fresh}$	$\langle \text{ref}(\text{raw-mut}, a, t), \Pi \rangle \Downarrow_a$ $(\Pi[a \mapsto x \# (\text{RawPtr}(\text{true}, u) :: i :: y)], u)$	SB-REF-RAW
$\Pi(a) = i :: y, \quad \text{tag}(i) = u,$ $i \neq \text{Own}(u)$	$\langle \text{die}(a, u), \Pi \rangle \Downarrow_a \Pi[a \mapsto y]$	SB-DIE
$\langle \text{op}(a + j, \dots), \Pi_j \rangle \Downarrow_a \Pi_{j+1} \text{ for } 0 \leq j < n,$ one fresh tag shared by all n cells	$\text{op}(\Pi_0, a, n, \dots) = \Pi_n$	SB-RANGE

In the mechanization the operations are `sb_own`, `sb_read`, `sb_write`, `sb_ref`, and `sb_die` in `src/obseq3/sb.lean`; the paper writes `useMut` for `sb_write` to keep the source-level reading.

2.2 MIRLite

MIRLite is a typed, sequential language for the memory-level part of Rust that matters to borrow reasoning (Fig. 2). A program is a list of statements. A statement assigns an expression to a place, or halts. A *place* is a local ℓ , a projection $p.q$ of a place along a path, or a dereference $*p$. Both p and d range over places; we write d when the place is the *destination* of an assignment, as in $d := e$, and p otherwise. An *expression* is a constant, a copy or a move of a place, or a reference to a place. A move is MIR’s move operand. As in Miri’s Stacked Borrows it has the semantics of a copy, a read of the source: that a moved-from place is not used again is enforced statically by Rust’s borrow checker and has no effect on the borrow stacks. The mechanization accordingly has one constructor for the two, and everything proved of copy holds of move. We do not model control flow in the main text; the guarded assignment of the full language is in Appendix A.

Types. A *layout* τ describes the shape of a memory region, and $|\tau|$ is its width in cells:

$$|\text{Nat}| = 1 \quad |\text{Ptr } \tau| = 1 \quad |(\tau_0, \dots, \tau_{n-1})| = \sum_{j=0}^{n-1} |\tau_j|.$$

In the pointer layout $\text{Ptr } \tau$, τ is the layout of the *pointee*, the region the pointer refers to. It is statically significant even though every pointer occupies one cell: it is what gives $*p$ a layout, and hence a width. Both σ and τ range over layouts; the choice records a role. When a judgment involves two layouts, σ is the region being traversed and τ the region selected from it. A *local* ℓ is a program variable, the counterpart of a MIR local such as `_0`. A context Γ lists the layouts of a program’s locals, a local is an index into that list, and $\ell : \tau$ means that entry ℓ of Γ is τ . We write

TERMS

Program $\equiv$ Stmt* halt
 Stmt $\ni s \equiv d := e \mid$ halt
 Place $\ni p \equiv \ell \mid p.q \mid *p$
 Place $\ni d$
 Local $\ni \ell = \mathbb{N}$
 Path $\ni q \equiv \varepsilon \mid f.q$
 $\mathbb{N} \ni f$
 Expr $\ni e \equiv \text{const}(w) \mid \text{copy}(p) \mid \text{move}(p)$
 $\mid \text{ref}(k, c, m, p)$
 Kind $\ni k \equiv \text{shared} \mid \text{mutable}$
 $\mid \text{raw-const} \mid \text{raw-mut}$
 $\mid \text{two-phase}$
 Flag $\ni c = \text{Bool}$
 Mask $\ni m = \text{Bool}^*$

SEMANTIC DOMAINS

State $\ni S = \mathbb{N} \times \text{Env} \times \text{Mem} \times \text{Perm}$
 Env $\ni E = \text{Local} \rightarrow \mathbb{N} \times \text{Tag}$
 Mem $\ni \mu = \mathbb{N} \rightarrow \text{Value}$
 Value $\ni v \equiv \text{undef} \mid \text{word}(w)$
 $\mid \text{ptr}(b, o, sz, t)$
 $\mathbb{N} \ni i, a, b, o, sz$
 Tag $\ni t, u = \mathbb{N}$
 Perm $\ni \Pi$

TYPES

Layout $\ni \tau \equiv \text{Nat} \mid \text{Ptr } \tau$
 $\mid (\tau_0, \dots, \tau_{n-1})$
 Layout $\ni \sigma$
 Ctx $\ni \Gamma = \text{Layout}^*$

Fig. 2. Grammar, semantic domains, and types of MIRLite. Every line is headed $X \ni x$: the metavariable x ranges over the set X . With $\equiv$ the line is a production: X is generated by the constructors on the right. With $=$ the set X is built from other sets: $\times$ is a product, $\rightarrow$ a partial map, and X^* the lists over X ; Perm is the set of permission states of Section 2.1. With nothing on the right it declares a further metavariable over a set defined elsewhere. Retag kinds k , flags c , and masks m are those of Section 2.1, repeated here. The main text covers these forms; Fig. 5 has the rest.

x and y for the locals 0 and 1 of the running program. A *path* q is a list of tuple-field indices, read left to right: ε selects the whole region and $f.q$ selects field f and then follows q . The rules use only two facts about a path. It is typed, $q : \sigma \rightarrow \tau$, meaning that following q through a σ -shaped region selects a τ -shaped one. And it has a cell offset $\text{off}(q)$, the total width of the fields it skips, with $\text{off}(q) + |r| \leq |\sigma|$. Places are indexed by the layout they select: $\Gamma \vdash \ell : \tau$ if $\ell : \tau \in \Gamma$; $\Gamma \vdash p.q : \tau$ if $\Gamma \vdash p : \sigma$ and $q : \sigma \rightarrow \tau$; and $\Gamma \vdash *p : \tau$ if $\Gamma \vdash p : \text{Ptr } \tau$. Expressions and statements are indexed the same way: $\text{const}(w)$ has layout Nat, $\text{copy}(p)$ and $\text{move}(p)$ the layout of p , $\text{ref}(k, c, m, p)$ the layout $\text{Ptr } \tau$ when $\Gamma \vdash p : \tau$, and $d := e$ requires d and e to have the same layout. For example, for a local $x : (\text{Nat}, (\text{Nat}, \text{Nat}))$ the place written $x.1.0$ is the nested place $(x.q_1).q_0$ with the one-field paths $q_1 = 1.\varepsilon$ and $q_0 = 0.\varepsilon$. Field 1 of x starts after the one cell of field 0, so $q_1 : (\text{Nat}, (\text{Nat}, \text{Nat})) \rightarrow (\text{Nat}, \text{Nat})$ with $\text{off}(q_1) = 1$; field 0 of that pair starts at once, so $q_0 : (\text{Nat}, \text{Nat}) \rightarrow \text{Nat}$ with $\text{off}(q_0) = 0$.

Reference formation. $\text{ref}(k, c, m, p)$ carries the retag kind k , protector flag c , and mask m of Section 2.1 and hands them unchanged to the permission model (E-REF). The kind is therefore not limited to Rust references: a MIRLite program can form raw pointers and two-phase borrows as well.

Semantic domains. A *cell value* v is undefined, a machine word w , or a pointer; $\bar{v}$ is a list of cell values and $\text{len}(\bar{v})$ its length. A pointer $\text{ptr}(b, o, sz, t)$ has four fields. The *base* b and *size* sz , in cells, identify the allocation $[b, b + sz)$ it points into, and every access through the pointer is checked against those bounds. The *offset* o locates the pointer inside that allocation, so the address it denotes is $b + o$; pointer arithmetic changes o and nothing else. The *tag* t is its provenance, the identity under which the permission model of Section 2.1 admits or rejects the access. Two pointers to the same address with different tags are different values, and only one of them may be allowed to write. A memory μ is a partial map from addresses to cell values together with the next bump-allocation address and the list of allocated ranges. Write $\mu(a)$ for the cell at a , which is undef when

Table 2. MIRLite semantics. $S.i$, $S.E$, $S.\mu$, and $S.\Pi$ are the components of a state $S = (i, E, \mu, \Pi)$, and $S[\Pi \mapsto \Pi']$ is S with its permission state replaced by Π' . Γ is the context of the program P being run; both are fixed throughout. All rules describe successful branches. An unbound local in a read, a malformed or out-of-bounds pointer, or a rejection by the permission model is an error and has no successor.

Premises	Conclusion	Rule Name
$S.E(\ell) = (a, t), \ell : \tau \in \Gamma$	$S \vdash \ell \Downarrow_p \langle (a, t, a, \tau), S.\Pi \rangle$	PLC-LOCAL
$S \vdash p \Downarrow_p \langle (a, t, b, sz), \Pi' \rangle$	$S \vdash p.q \Downarrow_p \langle (a + \text{off}(q), t, b, sz), \Pi' \rangle$	PLC-PROJ
$S \vdash p \Downarrow_p \langle (a, t, b, sz), \Pi_1 \rangle, b \leq a < b + sz,$ $\text{read}(\Pi_1, a, 1, t) = \Pi_2, S.\mu(a) = \text{ptr}(b', o', sz', t')$	$S \vdash *p \Downarrow_p \langle (b' + o', t', b', sz'), \Pi_2 \rangle$	PLC-DEREF
$\text{lookup}(S, d)$ is defined	$\text{prepare}(S, d) = S$	PREP-BOUND
$\text{lookup}(S, d)$ undefined, $\text{root}(d) = \ell, \ell : \tau \in \Gamma,$ $S.E(\ell) = \perp,$ $\text{alloc}(S.\mu, \tau) = (b, \mu'), \text{own}(S.\Pi, b, \tau) = (\Pi', t)$	$\text{prepare}(S, d) =$ $(S.i, S.E[\ell \mapsto (b, t)], \mu', \Pi')$	PREP-ALLOC
	$S \vdash \text{const}(w) \Downarrow_e ([\text{word}(w)], S)$	E-CONST
$S \vdash p \Downarrow_p \langle (a, t, b, sz), \Pi_1 \rangle, \Gamma \vdash p : \tau,$ $a + \tau \leq b + sz, \text{read}(\Pi_1, a, \tau , t) = \Pi_2$	$S \vdash \text{copy}(p) \Downarrow_e (S.\mu[a..a + \tau], S[\Pi \mapsto \Pi_2])$	E-COPY
$S \vdash \text{copy}(p) \Downarrow_e (\bar{v}, S')$	$S \vdash \text{move}(p) \Downarrow_e (\bar{v}, S')$	E-MOVE
$S \vdash p \Downarrow_p \langle (a, t, b, sz), \Pi_1 \rangle, \Gamma \vdash p : \tau,$ $a + \tau \leq b + sz, \text{ref}(\Pi_1, a, \tau , t, k, c, m) = (\Pi_2, u)$	$S \vdash \text{ref}(k, c, m, p) \Downarrow_e$ $([\text{ptr}(b, a - b, sz, u)], S[\Pi \mapsto \Pi_2])$	E-REF
$P(S.i) = (d := e), \text{prepare}(S, d) = S_1,$ $S_1 \vdash e \Downarrow_e (\bar{v}, S_2), S_2 \vdash d \Downarrow_p \langle (a, t, b, sz), \Pi_3 \rangle,$ $a + \text{len}(\bar{v}) \leq b + sz, \text{useMut}(\Pi_3, a, \text{len}(\bar{v}), t) = \Pi_4$	$S \xrightarrow{\text{mir}}$ $(S.i + 1, S_2.E, S_2.\mu[a \mapsto \bar{v}], \Pi_4)$	ASSGN
$P(S.i) = \text{halt}$ or $P(S.i) = \perp$	$S \xrightarrow{\text{mir}} S$	HALT

the map has no entry there; $\mu[a..a + n]$ for the list of n cells from a ; and $\mu[a \mapsto \bar{v}]$ for the memory with the cells from a replaced by the list $\bar{v}$. The environment E maps a local either to no binding or to an allocation base and its owning tag. A state is $S = (i, E, \mu, \Pi)$ with i the program counter. Table 2 gives the rules; they use three judgments, which we introduce before the interpreter.

Place resolution ($\Downarrow_p$). The judgment $S \vdash p \Downarrow_p \langle (a, t, b, sz), \Pi' \rangle$ reads: in state S , the well-typed place p resolves to $\langle a, t, b, sz \rangle$, and the permission state becomes Π' . The four components of a *resolved place* are the address a the place denotes, the tag t through which it is to be accessed, and the base b and size sz of the allocation it lies in. It is to a place what a list of cell values is to an expression, the result of evaluating it, and it is never stored. The state is on the left because resolution consults all of it: the environment for a local, memory and permissions for a dereference. It does not write memory, but it can change the permission state. **PLC-LOCAL** reads the environment. **PLC-PROJ** is typed address arithmetic: it preserves provenance, bounds, and permissions. **PLC-DEREF** changes provenance, because the pointer *value*, not the place holding it, identifies the referent; loading that value is a one-cell read through the tag of the place that holds it. Nested dereferences thread these permission changes from the inside out. A separate *pure lookup*, $\text{lookup}(S, p)$, follows the same address calculation but only inspects the pointer cell, with neither a bounds check nor a read. It is used only to decide whether an assignment root must be allocated.

Expression evaluation ($\Downarrow_e$). The judgment $S \vdash e \Downarrow_e (\bar{v}, S')$ produces exactly $|\tau|$ cells for an expression of layout τ . **E-COPY** and **E-REF** resolve their place and then act on the entire selected range, and **E-MOVE** is **E-COPY**: one read, or one ref, of width $|\tau|$. Reading an absent but permitted

cell yields undef. The pointer built by **E-REF** carries the fresh tag u and the allocation's base and size, so later bounds checks are against the whole allocation.

Interpreter ($\rightarrow_{\text{mir}}$). A step fetches $P(i)$. **ASSGN** runs four phases in a fixed order: prepare the destination's root, evaluate the expression, resolve the destination, write. Preparation (**PREP-BOUND**, **PREP-ALLOC**) allocates the root local of d when it is unbound: the bump allocator returns a fresh base b , own a fresh tag t , and E is extended. A destination rooted through a dereference is never allocated implicitly; that case is an error. Crucially, *the whole expression is evaluated before the destination is resolved*. Hence $\text{copy}(p)$ materializes its cells before a destination access can invalidate a tag that p needs. The conclusion takes its environment and memory from S_2 , the state after the expression, so allocation during preparation and any effect of the expression survive. We write $S \xrightarrow[n]{P} S'$ for n steps; **halt** and a missing statement are fixed points (**HALT**), so extra fuel is harmless.

Example. The running program of this paper declares a nested tuple x and a pointer y , and is small enough to execute by hand:

```
MIRLITE   $\Gamma = [x : (\text{Nat}, (\text{Nat}, \text{Nat})), y : \text{Ptr Nat}]$ 
0:  $x.0 := \text{const}(5)$ 
1:  $x.1.0 := \text{const}(42)$ 
2:  $y := \text{ref}(\text{mutable}, \text{false}, [], x.1.0)$ 
3:  $*y := \text{const}(7)$ 
4: halt
```

Statement 0 allocates x , because an assignment to an unbound root allocates it. Statement 1 writes one cell in the middle of x ; it is the statement for which the compiler of Section 2.4 must introduce, and then retire, a tag the source never sees. Statement 2 allocates y and stores a mutable reference to the cell just written. Statement 3 writes through that reference. Execution starts from the empty state: the bump allocator hands out addresses from 0 and tags are minted from 1 upward, tag 0 being reserved (Appendix A.2). The program thus touches four cells: x has width 3 and occupies addresses 0 to 2, holding $x.0$, $x.1.0$, and $x.1.1$, and y has width 1 and occupies address 3. All concrete addresses, tags, registers, and labels in this section are those of the mechanized semantics running this program.

Table 3. The MIRLite configuration after every statement of the running program. A cell column gives the cell's value over its borrow stack; “—” is an unallocated cell, shading marks what the statement changed, and “next” is $\Pi.\text{NextTag}$.

after statement	E	cell 0 $x.0$	cell 1 $x.1.0$	cell 2 $x.1.1$	cell 3 y	next
initially	—	—	—	—	—	1
0: $x.0 := \text{const}(5)$	$x \mapsto (0, 1)$	word(5) [Own(1)]	undef [Own(1)]	undef [Own(1)]	—	2
1: $x.1.0 := \text{const}(42)$	$x \mapsto (0, 1)$	word(5) [Own(1)]	word(42) [Own(1)]	undef [Own(1)]	—	2
2: $y := \text{ref}(\text{mutable}, \dots, x.1.0)$	$x \mapsto (0, 1)$ $y \mapsto (3, 2)$	word(5) [Own(1)]	word(42) [MutRef(3), Own(1)]	undef [Own(1)]	ptr(0, 1, 3, 3) [Own(2)]	4
3: $*y := \text{const}(7)$	$x \mapsto (0, 1)$ $y \mapsto (3, 2)$	word(5) [Own(1)]	word(7) [MutRef(3), Own(1)]	undef [Own(1)]	ptr(0, 1, 3, 3) [Own(2)]	4
4: halt	$x \mapsto (0, 1)$ $y \mapsto (3, 2)$	word(5) [Own(1)]	word(7) [MutRef(3), Own(1)]	undef [Own(1)]	ptr(0, 1, 3, 3) [Own(2)]	4

Table 3 gives the whole configuration after each statement; the program counter, not shown, is $k + 1$ after statement k , and `halt` leaves it at 4. Read a row against the one above it: the shaded entries are exactly what the rules below change.

- 0 x is unbound, so **PREP-ALLOC** allocates the three cells $[0, 3)$, binds $x \mapsto (0, 1)$, and **SB-OWN** gives each cell the stack $[\text{Own}(1)]$. **E-CONST** produces $[\text{word}(5)]$. **PLC-LOCAL** and **PLC-PROJ** resolve $x.0$ to $\langle 0, 1, 0, 3 \rangle$. **ASSGN** then writes cell 0 after a `useMut` through tag 1, which **SB-USE-MUT** accepts without changing the stack. Cells 1 and 2 are allocated but still `undef`.
- 1 Preparation is **PREP-BOUND**: nothing is allocated. Two applications of **PLC-PROJ** resolve $(x.q_1).q_0$ to $\langle 1, 1, 0, 3 \rangle$; the inner, zero-offset projection changes neither the address nor the tag. **ASSGN** writes cell 1 through the owner's tag. There is no source retag, so the stack of cell 1 does not change, and the sibling cell 2, $x.1.1$, is untouched.
- 2 **PREP-ALLOC** allocates y at address 3 with owning tag 2, *before* the expression is evaluated. **E-REF** resolves $x.1.0$ as in statement 1, and **SB-REF-MUT** pushes `MutRef(3)` on cell 1 only. The pointer `ptr(0, 1, 3, 3)` is stored in y . The tag counter moves from 2 to 4: one tag for y , one for the reference.
- 3 **PLC-DEREF** resolves y to $\langle 3, 2, 3, 1 \rangle$, performs a one-cell read of cell 3 through tag 2 (**SB-READ**, which changes nothing here), and returns $\langle 1, 3, 0, 3 \rangle$, the address and tag *loaded from memory*. **ASSGN** writes 7 through tag 3, which is on top of cell 1's stack, so that stack does not change either.
- 4 **HALT**: the configuration is a fixed point.

Formalization of the typed syntax and source transition system: `src/obseq3/syntax.lean` and `src/obseq3/mirlite_semantics.lean`.

2.3 OSEA-IR

OSEA-IR is a register machine that exposes the memory and permission events implicit in a MIRLite statement (Fig. 3). A source place is gone: an instruction names a register holding a concrete pointer, an offset, and a number of cells. Consequently one source step may need several

TERMS

Program $\ni Q = \mathbb{N} \rightarrow \text{Instr}$
 Instr $\ni I ::= r := h$
 | $\text{store}_\theta(r_s, r_p)$
 | $\text{storec}_\theta(\bar{v}, r_p)$
 | $\text{die}(r, n)$
 | halt
 Rhs $\ni h ::= \text{load}_\theta(r)$
 | alloc_θ
 | $\text{borrow}(k, c, m, n, r, \delta)$
 Register $\ni r ::= R_0 \mid R_1 \mid \dots$

SEMANTIC DOMAINS

State $\ni T = \mathbb{N} \times \text{RegFile} \times \text{Mem} \times \text{Perm}$
 RegFile $\ni R = \text{Register} \rightarrow \text{Type} \times \text{Value}^*$
 Mem $\ni \mu_t = \mathbb{N} \rightarrow \text{Value}$
 Value $\ni v ::= \text{undef} \mid \text{dat}(w)$
 | $\text{ptr}(b, o, sz, t)$
 Type $\ni \theta ::= \text{NatTy} \mid \text{PTy}$
 | $\text{TupTy}([\theta_0, \dots, \theta_{n-1}])$

COMPILER STATE

CState $\ni C = \mathbb{N} \times \mathbb{N} \times \text{Code} \times \text{LocalMap}$
 Code $\ni K = \mathbb{N} \rightarrow \text{Instr}$
 LocalMap $\ni L = \text{Local} \rightarrow \text{Register} \times \text{Layout}$
 Cleanup $\ni D = (\text{Register} \times \mathbb{N})^*$

Fig. 3. Grammar and semantic domains of OSEA-IR (left, top right), and the state of the compiler (bottom right). Fig. 5 has the remaining instructions.

target steps and may create short-lived tags with no source counterpart. A program Q is a *partial* map from labels to instructions, which lets the compiler extend code without renumbering an earlier fragment. Instructions are register assignments $r := h$, stores of a register or of a constant through a pointer, the retirement die of a borrow, and halt. A right-hand side h loads through a pointer, allocates, or borrows.

Semantic domains. Target values mirror source values, with `dat` for a machine word; the pointer fields have the same meaning as at the source. A register holds a runtime type and an *entire cell list*; the register file is a finite shadowing map. Runtime types are distinct from layouts. The erasure $[\tau]$ maps a layout to a runtime type,

$$[\text{Nat}] = \text{NatTy} \quad [\text{Ptr } \tau] = \text{PTy} \quad [(\tau_0, \dots, \tau_{n-1})] = \text{TupTy}([\tau_0], \dots, [\tau_{n-1}]),$$

so OSEA-IR records that a register holds a pointer but not the pointee's layout, and $\text{typeSize}([\tau]) = |\tau|$. A target state is $T = (j, R, \mu_t, \Pi_t)$; memory carries a bump-allocation watermark and an allocation table as at the source, and the permission state is the same Stacked Borrows instance. Table 4 gives the rules.

RHS evaluation ($\Downarrow_h$). The judgment $T \vdash h \Downarrow_h (\theta, \bar{v}, T')$ evaluates a right-hand side to a runtime type, a cell list, and a state. It may update memory or permissions but does not advance j . **RVAL-LOAD** checks the complete range and then reads it through the pointer's tag. **RVAL-BORROW** retags n cells at offset δ from the pointer and returns the same pointer, moved by δ and carrying the fresh tag; offsets are natural numbers, so there is no negative-offset case, and a zero-length

Table 4. OSEA-IR semantics. $T.R$, $T.\mu_t$, and $T.\Pi_t$ are components of a state $T = (j, R, \mu_t, \Pi_t)$, and $T[\dots \mapsto \dots]$ replaces components, as in Table 2; the instruction rules write the state out as a tuple instead. Q is the program being run. In every rule that reads $R(r)$, the register must hold exactly one pointer; its stored runtime type θ_r is immaterial.

Premises	Conclusion	Rule Name
$T.R(r) = (\theta_r, [\text{ptr}(b, o, sz, t)]), \quad a = b + o,$ $a + \text{typeSize}(\theta) \leq b + sz,$ $\text{read}(T.\Pi_t, a, \text{typeSize}(\theta), t) = \Pi'_t$	$T \vdash \text{load}_\theta(r) \Downarrow_h$ $(\theta, T.\mu_t[a..a + \text{typeSize}(\theta)], T[\Pi_t \mapsto \Pi'_t])$	RVAL-LOAD
$n = \text{typeSize}(\theta), \quad \text{alloc}(T.\mu_t, n) = (b, \mu'_t),$ $\text{own}(T.\Pi_t, b, n) = (\Pi'_t, u)$	$T \vdash \text{alloc}_\theta \Downarrow_h$ $(\text{PTy}, [\text{ptr}(b, 0, n, u)], T[\mu_t \mapsto \mu'_t, \Pi_t \mapsto \Pi'_t])$	RVAL-ALLOC
$T.R(r) = (\theta_r, [\text{ptr}(b, o, sz, t)]), \quad a = b + o + \delta,$ $a + n \leq b + sz, \quad \text{ref}(T.\Pi_t, a, n, t, k, c, m) = (\Pi'_t, u)$	$T \vdash \text{borrow}(k, c, m, n, r, \delta) \Downarrow_h$ $(\text{PTy}, [\text{ptr}(b, o + \delta, sz, u)], T[\Pi_t \mapsto \Pi'_t])$	RVAL-BORROW
$Q(j) = (r := h),$ $(j, R, \mu_t, \Pi_t) \vdash h \Downarrow_h (\theta, \bar{v}, (j, R, \mu_1, \Pi_1))$	$(j, R, \mu_t, \Pi_t) \xrightarrow{\text{sea}}$ $(j + 1, R[r \mapsto (\theta, \bar{v})], \mu_1, \Pi_1)$	EXEC-ASSGN
$Q(j) = \text{store}_\theta(r_s, r_p), \quad R(r_s) = (\theta, \bar{v}),$ $R(r_p) = (\theta_p, [\text{ptr}(b, o, sz, t)]), \quad a = b + o,$ $a + \text{len}(\bar{v}) \leq b + sz, \quad \text{useMut}(\Pi_t, a, \text{len}(\bar{v}), t) = \Pi'_t$	$(j, R, \mu_t, \Pi_t) \xrightarrow{\text{sea}}$ $(j + 1, R, \mu_t[a \mapsto \bar{v}], \Pi'_t)$	EXEC-STORE
$Q(j) = \text{store}_\theta(\bar{v}, r_p), \quad \text{len}(\bar{v}) = \text{typeSize}(\theta),$ $R(r_p) = (\theta_p, [\text{ptr}(b, o, sz, t)]), \quad a = b + o,$ $a + \text{len}(\bar{v}) \leq b + sz, \quad \text{useMut}(\Pi_t, a, \text{len}(\bar{v}), t) = \Pi'_t$	$(j, R, \mu_t, \Pi_t) \xrightarrow{\text{sea}}$ $(j + 1, R, \mu_t[a \mapsto \bar{v}], \Pi'_t)$	EXEC-STOREC
$Q(j) = \text{die}(r, n), \quad R(r) = (\theta_r, [\text{ptr}(b, o, sz, t)]),$ $\text{die}(\Pi_t, b + o, n, t) = \Pi'_t$	$(j, R, \mu_t, \Pi_t) \xrightarrow{\text{sea}} (j + 1, R, \mu_t, \Pi'_t)$	EXEC-DIE
$Q(j) = \text{halt} \text{ or } Q(j) = \perp$	$T \xrightarrow{\text{sea}} T$	EXEC-HALT

borrow at one past the end is legal. **RVAL-ALLOC** is the only rule of the main text that changes both memory and permissions.

Interpreter ($\rightarrow_{\text{sea}}$). **EXEC-ASSGN** evaluates its right-hand side *before* inserting the returned entry. The two stores share one write-through-pointer helper: extract one pointer from r_p , check the range, useMut, write, advance. They differ in where the cells come from: **EXEC-STORE** checks that r_s holds exactly the runtime type θ , and **EXEC-STOREC** that the constant has the right length. **EXEC-DIE** has no separate range check: its admissibility is exactly that of the permission operation. There is no global well-formedness premise on register files; the instruction that consumes a register checks the shape it needs, which keeps malformed programs executable with explicit errors. We write $T \xrightarrow[n]{Q} T'$ for exactly n steps. Iteration does not stop early at a fixed point (**EXEC-HALT**), which makes the target step count of the simulation existential without a separate reflexive-transitive closure.

Example. Table 5 executes the code the compiler emits for the running program (Fig. 4), one group of labels per source statement. Compare each group’s last row with the corresponding row of Table 3: the cells agree up to word/dat, the stacks agree up to the tags, and the tag counter runs ahead.

Table 5. The OSEA-IR configuration after every instruction of the compiled running program, in the format of Table 3. The R column shows the binding the instruction adds; registers are never removed, so the register file is the union of the rows above. After label k the program counter is $k + 1$. $\theta_x = \text{TupleTy}([\text{NatTy}, \text{TupleTy}([\text{NatTy}, \text{NatTy}]]); \text{mut}$ abbreviates mutable, false, []. Labels 0–1, 2–4, 5–7, 8–9, and 10 are source statements 0 to 4.

after instruction	R gains	cell 0 $x.0$	cell 1 $x.1.0$	cell 2 $x.1.1$	cell 3 y	next
<i>initially</i>	—	—	—	—	—	1
0: $R_0 := \text{alloc}_{\theta_x}$	$R_0 \mapsto \text{ptr}(0, 0, 3, 1)$	undef [Own(1)]	undef [Own(1)]	undef [Own(1)]	—	2
1: $\text{storec}_{\text{NatTy}}([\text{dat}(5)], R_0)$	—	dat(5) [Own(1)]	undef [Own(1)]	undef [Own(1)]	—	2
2: $R_1 := \text{borrow}(\text{mut}, 1, R_0, 1)$	$R_1 \mapsto \text{ptr}(0, 1, 3, 2)$	dat(5) [Own(1)]	undef [MutRef(2), Own(1)]	undef [Own(1)]	—	3
3: $\text{storec}_{\text{NatTy}}([\text{dat}(42)], R_1)$	—	dat(5) [Own(1)]	dat(42) [MutRef(2), Own(1)]	undef [Own(1)]	—	3
4: $\text{die}(R_1, 1)$	—	dat(5) [Own(1)]	dat(42) [Own(1)]	undef [Own(1)]	—	3
5: $R_2 := \text{alloc}_{\text{PTy}}$	$R_2 \mapsto \text{ptr}(3, 0, 1, 3)$	dat(5) [Own(1)]	dat(42) [Own(1)]	undef [Own(1)]	undef [Own(3)]	4
6: $R_3 := \text{borrow}(\text{mut}, 1, R_0, 1)$	$R_3 \mapsto \text{ptr}(0, 1, 3, 4)$	dat(5) [Own(1)]	dat(42) [MutRef(4), Own(1)]	undef [Own(1)]	undef [Own(3)]	5
7: $\text{store}_{\text{PTy}}(R_3, R_2)$	—	dat(5) [Own(1)]	dat(42) [MutRef(4), Own(1)]	undef [Own(1)]	ptr(0, 1, 3, 4) [Own(3)]	5
8: $R_4 := \text{load}_{\text{PTy}}(R_2)$	$R_4 \mapsto \text{ptr}(0, 1, 3, 4)$	dat(5) [Own(1)]	dat(42) [MutRef(4), Own(1)]	undef [Own(1)]	ptr(0, 1, 3, 4) [Own(3)]	5
9: $\text{storec}_{\text{NatTy}}([\text{dat}(7)], R_4)$	—	dat(5) [Own(1)]	dat(7) [MutRef(4), Own(1)]	undef [Own(1)]	ptr(0, 1, 3, 4) [Own(3)]	5
10: halt	—	dat(5) [Own(1)]	dat(7) [MutRef(4), Own(1)]	undef [Own(1)]	ptr(0, 1, 3, 4) [Own(3)]	5

391

392 0–1 **RVAL-ALLOC** allocates $[0, 3)$ and issues **SB-OWN**, so R_0 holds the owner’s pointer with tag 1.

393 **EXEC-STOREC** writes cell 0 through it. The state equals the source’s after statement 0.

394 2–4 One source write takes three instructions, and their permission trace is Fig. 1 at $a = 1, t = 1,$

395 $u = 2$. **RVAL-BORROW** issues **SB-REF-MUT** on cell 1 alone and mints tag 2; **EXEC-STOREC** writes

396 through tag 2; **EXEC-DIE** issues **SB-DIE**, after which cell 1’s stack is again $[\text{Own}(1)]$. The memory

397 and the stacks now equal the source’s after statement 1, but the tag counter is 3 where the

398 source’s is 2.

399 5–7 **RVAL-ALLOC** allocates y , **RVAL-BORROW** mints the reference, and **EXEC-STORE** stores it.

400 Because the counter ran ahead, y ’s owning tag is 3 here and 2 at the source, and the stored

401 reference carries tag 4 here and 3 at the source. Nothing is retired: this borrow is the program’s

402 own.

403 8–9 **RVAL-LOAD** reads cell 3 through tag 3 and leaves $\text{ptr}(0, 1, 3, 4)$ in R_4 ; **EXEC-STOREC** writes cell

404 1 through the loaded tag 4.

405 10 **EXEC-HALT**. R_1 still holds a pointer with the dead tag 2; registers are not observable, so the

406 simulation of Section 2.5 ignores it.

407 Formalization of target values, RHS evaluation, instruction stepping, and fuelled execution: `src/obseq3/oseair.lean`.

408 2.4 Compiler

409 The compiler is a state-passing translation from typed MIRLite syntax to OSEA-IR code. Its oblig-
 410 ation is to preserve the source evaluation order while introducing explicit loads, retags, stores, and
 411 the retirement of the tags it introduces. A compiler state is $C = (n_r, n_l, K, L)$ (Fig. 3, bottom right):
 412 n_r and n_l are the next fresh register and label, K is the emitted code map, and L maps a source local
 413 to a register and a source layout; code uses the layout’s erasure. Write $C \oplus [I_0, \dots, I_{n-1}]$ for the state
 414 that installs the list at labels $[n_l, n_l + n)$ and advances n_l by n , and “ r fresh in C ” for $r = R_{n_r}$ with
 415 n_r advanced. Every compiler computation is monotone: counters do not decrease, code below the
 416 old n_l is unchanged, and every old entry of L remains. A cleanup list D records borrows to retire;
 417 $\text{cleanup}(D)$ reverses D and maps each entry (r, n) to $\text{die}(r, n)$. Table 6 gives the rules.

418 **Establishing roots.** Writing an unbound root must mirror source preparation. **ROOT-BOUND**
 419 emits nothing; **ROOT-ALLOC** allocates the root local of the destination and records its register. For
 420 a dereference the root is the pointer local. The source never allocates a pointee implicitly and
 421 fails instead, so a forward simulation never observes that case. Read lowering does not allocate
 422 a missing local: resolving a read requires that the root already exist, and the compiler is *checked*,
 423 returning an error for such a program.

424 **Place-to-register compilation** ($\Downarrow_{\text{CP}}$). The judgment $C \vdash_k p \Downarrow_{\text{CP}} (r, D, C')$ lowers place p ,
 425 leaves a pointer to it in r , and returns a cleanup list. Its index k is the retag kind of any borrow
 426 the lowering emits: reads lower their place with $k = \text{shared}$ and assignment destinations with $k =$
 427 mutable . **PLACE-LOCAL** is a lookup. A zero-offset projection reuses the base pointer (**PLACE-PROJ-0**):
 428 it denotes the same address and needs no narrower pointer. A nonzero projection emits a borrow
 429 of the *final selected width* (**PLACE-PROJ-N**). A dereference loads the pointer cell (**PLACE-DEREF**) and
 430 retires the borrows that were needed to reach that cell, but it returns an empty cleanup list: the
 431 loaded tag belongs to the source program and must not be retired.

Table 6. Compilation rules. $\text{root}(d)$ is the local reached from d through projections and dereferences. In **PLACE-PROJ-0** and **PLACE-PROJ-N** the base b is not itself a projection (**PLACE-ASSOC** applies first), and the selected place has layout τ .

Premises	Conclusion	Rule Name
$\text{root}(d) = \ell, L(\ell) \text{ defined}$	$C \vdash \text{root}(d) \rightarrow C$	ROOT-BOUND
$\text{root}(d) = \ell : \tau, L(\ell) = \perp, r \text{ fresh in } C$	$C \vdash \text{root}(d) \rightarrow (C \oplus [r := \text{alloc}_{ \tau }]) [L(\ell) \mapsto (r, \tau)]$	ROOT-ALLOC
$L(\ell) = (r, \tau)$	$C \vdash_k \ell \Downarrow_{\text{cP}} (r, [], C)$	PLACE-LOCAL
$C \vdash_k b.(q \cdot p) \Downarrow_{\text{cP}} (r, D, C')$	$C \vdash_k (b.q).p \Downarrow_{\text{cP}} (r, D, C')$	PLACE-ASSOC
$\text{off}(q) = 0, C \vdash_k b \Downarrow_{\text{cP}} (r_b, D_b, C')$	$C \vdash_k b.q \Downarrow_{\text{cP}} (r_b, D_b, C')$	PLACE-PROJ-0
$\delta = \text{off}(q) > 0, C \vdash_k b \Downarrow_{\text{cP}} (r_b, D_b, C_1),$ $r_f \text{ fresh in } C_1,$ $I = (r_f := \text{borrow}(k, \text{false}, [], \tau , r_b, \delta))$	$C \vdash_k b.q \Downarrow_{\text{cP}} (r_f, D_b \uplus [(r_f, \tau)], C_1 \oplus [I])$	PLACE-PROJ-N
$C \vdash_{\text{shared}} p \Downarrow_{\text{cP}} (r_p, D_p, C_1), r \text{ fresh in } C_1$	$C \vdash_k *p \Downarrow_{\text{cP}} (r, [], C_1 \oplus [r := \text{load}_{\text{PTy}}(r_p)] \oplus \text{cleanup}(D_p))$	PLACE-DEREF
$L(\ell) = (r_b, \tau), r \text{ fresh in } C$	$C \vdash_{k,c,m} \ell \Downarrow_{\text{cB}} (r, C \oplus [r := \text{borrow}(k, c, m, \tau , r_b, 0)])$	BPLACE-LOCAL
$C \vdash_k b \Downarrow_{\text{cP}} (r_b, D_b, C_1), r \text{ fresh in } C_1$	$C \vdash_{k,c,m} b.q \Downarrow_{\text{cB}} (r, C_1 \oplus [r := \text{borrow}(k, c, m, \tau , r_b, \text{off}(q))])$	BPLACE-PROJ
$C \vdash_{\text{shared}} p \Downarrow_{\text{cP}} (r_p, D_p, C_1),$ $r_\eta, r \text{ fresh in } C_1$	$C \vdash_{k,c,m} *p \Downarrow_{\text{cB}} (r, C_1 \oplus [\eta := \text{load}_{\text{PTy}}(r_p)] \oplus \text{cleanup}(D_p) \oplus [r := \text{borrow}(k, c, m, \tau , r_\eta, 0)])$	BPLACE-DEREF
	$C \vdash \text{const}(w) \Downarrow_{\text{cR}} (\lambda r_d. [\text{storec}_{\text{NatTy}}([\text{dat}(w)], r_d)], C)$	RHS-CONST
$C \vdash_{\text{shared}} p \Downarrow_{\text{cP}} (r_s, D_s, C_1), p : \tau,$ $r_v \text{ fresh in } C_1$	$C \vdash \text{copy}(p) \Downarrow_{\text{cR}} (\lambda r_d. [\text{store}_{ \tau }(r_v, r_d)], C_1 \oplus [r_v := \text{load}_{ \tau }(r_s)] \oplus \text{cleanup}(D_s))$	RHS-COPY
$C \vdash \text{copy}(p) \Downarrow_{\text{cR}} (F, C')$	$C \vdash \text{move}(p) \Downarrow_{\text{cR}} (F, C')$	RHS-MOVE
$C \vdash_{k,c,m} p \Downarrow_{\text{cB}} (r_v, C_1)$	$C \vdash \text{ref}(k, c, m, p) \Downarrow_{\text{cR}} (\lambda r_d. [\text{store}_{\text{PTy}}(r_v, r_d)], C_1)$	RHS-REF
$C \vdash \text{root}(d) \rightarrow C_1, C_1 \vdash e \Downarrow_{\text{cR}} (F, C_2),$ $C_2 \vdash_{\text{mutable}} d \Downarrow_{\text{cP}} (r_d, D_d, C_3)$	$C \vdash d := e \rightarrow_c C_3 \oplus F(r_d) \oplus \text{cleanup}(D_d)$	STMT-ASSIGN
	$C \vdash \text{halt} \rightarrow_c C \oplus [\text{halt}]$	STMT-HALT
	$C \vdash [] \rightarrow_p C$	PROG-EMPTY
$C \vdash s \rightarrow_c C_1, C_1 \vdash P \rightarrow_p C'$	$C \vdash s :: P \rightarrow_p C'$	PROG-STEP

Definition 2.1 (Route borrow) A route borrow is the borrow instruction emitted by **PLACE-PROJ-N** to obtain a pointer to a projected subregion of a place that the compiled code already holds a pointer to. Its fresh tag is a route tag. A route borrow is unprotected, has an empty mask, and spans exactly the width of the layout selected by the place. Its register and width are recorded as an entry (r, n) of the cleanup list D , and the tag is retired by $\text{die}(r, n)$ before the next source-statement boundary. A route tag is never stored to memory by the compiled code and has no source counterpart.

Route tags are the only tags the compiled code mints that the source does not. Every other target retag corresponds to a source ref and produces a tag that the source program also stores.

Flattening. A place may contain nested projections, but emitting a borrow for each syntactic layer would change the accessed ranges. **PLACE-ASSOC** fuses adjacent paths before any borrow is emitted; it is the on-the-fly form of

```
def projInto : Place  $\Gamma$   $\sigma \rightarrow$  PathTo  $\sigma$   $\tau \rightarrow$  Place  $\Gamma$   $\tau$ 
  | .proj b q, p => .proj b (q.append p)
  | b,          p => .proj b p
```

```
def flattenPlace : Place  $\Gamma$   $\tau \rightarrow$  Place  $\Gamma$   $\tau$ 
  | .local l      => .local l
  | .deref p      => .deref (flattenPlace p)
  | .proj b path => projInto (flattenPlace b) path
```

where, for $q : \sigma_0 \rightarrow \sigma$ and $p : \sigma \rightarrow \tau$, the composed path satisfies $q \cdot p : \sigma_0 \rightarrow \tau$ and $\text{off}(q \cdot p) = \text{off}(q) + \text{off}(p)$. Flattening stops at a dereference, because an offset cannot be moved across a memory-loaded pointer. It preserves the selected layout and the source resolution result. For x.1.0, lowering without **PLACE-ASSOC** would borrow the two-cell field x.1 and then reuse its zero-offset subfield, touching cells 1 and 2. With it, the compiler emits one borrow of width one at the composed offset one and leaves the sibling x.1.1 alone.

Escaping borrows ($\Downarrow_{\text{CB}}$). Reference formation needs the distinct judgment $C \vdash_{k,c,m}$ $p \Downarrow_{\text{CB}} (r, C')$. It follows the same path normalization but *always* ends in a borrow($k, c, m, |\tau|, \dots$), including at offset zero (**BPLACE-LOCAL**, **BPLACE-PROJ**), and for a dereference it first loads the stored pointer and then borrows the pointed-to range (**BPLACE-DEREF**). Its final tag is not a route tag and is never retired: it escapes in the pointer value the program stores, and the correctness proof extends the source-to-target tag map with it.

Expression-to-instructions compilation ($\Downarrow_{\text{CR}}$). The judgment $C \vdash e \Downarrow_{\text{CR}} (F, C')$ emits all the work that precedes the destination and returns a *store function* F that awaits the destination register. **RHS-CONST** emits nothing. **RHS-COPY**, and with it **RHS-MOVE**, materializes the copied cells in a register *before* the destination is lowered. This is what realizes the source rule's order, evaluate, then resolve the destination, then write, when both resolutions have observable permission effects. **RHS-REF** retains the escaping tag. Every expression of the full language has the same shape (Appendix A).

Compilation judgments ($\rightarrow_c, \rightarrow_p$). **STMT-ASSIGN** is the only assignment rule, for every destination: the emitted intervals concatenate as

$$\text{root}(d); \text{ pre-code of } e; \text{ lowering of } d; F(r_d); \text{ cleanup}(D_d).$$

PROG-EMPTY and **PROG-STEP** fold statement compilation from left to right from the initial state $C_{\text{init}} = (0, 0, \emptyset, \emptyset)$; the target program is the final code map, or the first compiler error. For source statement index i , compiling the prefix $P[0..i]$ from C_0 produces a state C_i whose next label $C_i.n_l$ is the target entry label of statement i . Monotonicity gives the two facts simulation needs,

$$C_i.n_l \leq C_{i+1}.n_l \quad \wedge \quad \forall j < C_i.n_l. C_{i+1}.K(j) = C_i.K(j),$$

so compiling later statements cannot change a fragment already emitted. No fixed instruction-per-statement ratio is assumed.

Example. We compile the running program from C_{init} (Fig. 4), tracking (n_r, n_l, L) . Statement 0: $L(x) = \perp$, so **ROOT-ALLOC** emits label 0 and sets $L(x) = (R_0, \tau_x)$; **RHS-CONST** has no pre-code; x.0 has offset zero, so **PLACE-PROJ-0** and **PLACE-LOCAL** return $(R_0, [])$ and emit nothing; $F(R_0)$ is label

MIRLITE	OSEA-IR
0: $x.0 := \text{const}(5)$	0: $R_0 := \text{alloc}_{\theta_x}$ 1: $\text{storec}_{\text{NatTy}}([\text{dat}(5)], R_0)$
1: $x.1.0 := \text{const}(42)$	2: $R_1 := \text{borrow}(\text{mutable}, \text{false}, [], 1, R_0, 1)$ 3: $\text{storec}_{\text{NatTy}}([\text{dat}(42)], R_1)$ 4: $\text{die}(R_1, 1)$
2: $y := \text{ref}(\text{mutable}, \text{false}, [], x.1.0)$	5: $R_2 := \text{alloc}_{\text{PTy}}$ 6: $R_3 := \text{borrow}(\text{mutable}, \text{false}, [], 1, R_0, 1)$ 7: $\text{store}_{\text{PTy}}(R_3, R_2)$
3: $*y := \text{const}(7)$	8: $R_4 := \text{load}_{\text{PTy}}(R_2)$ 9: $\text{storec}_{\text{NatTy}}([\text{dat}(7)], R_4)$
4: halt	10: halt

Fig. 4. Compilation of the running program. Each row is one source statement and the labelled instructions emitted for it.

1. The state is $(1, 2, \{x \mapsto R_0\})$. Statement 1: **ROOT-BOUND**; **PLACE-ASSOC** rewrites $(x.q_1).q_0$ to $x.(q_1 \cdot q_0)$ with offset $1 + 0 = 1$ and selected width $|\text{Nat}| = 1$; **PLACE-PROJ-N** emits the route borrow at label 2 and returns $(R_1, [(R_1, 1)])$; the store is label 3; the cleanup is label 4. Every number is derived: path composition gives the offset, the selected layout the borrow length, erasure gives NatTy , and reversing the singleton cleanup list gives the die. Statement 2: **ROOT-ALLOC** emits label 5 for y before the expression, mirroring **PREP-ALLOC**; **RHS-REF** uses **BPLACE-PROJ**, whose borrow at label 6 has the same shape as the route borrow of label 2 but is recorded in no cleanup list; label 7 stores it. There is no die. Statement 3: **PLACE-DEREF** lowers y by **PLACE-LOCAL**, emits the load at label 8, and returns $(R_4, [])$; label 9 stores through the loaded pointer. There is again no die: retiring R_4 's tag would pop the program's own reference. The final state is $(5, 11, \{x \mapsto R_0, y \mapsto R_2\})$, and its code map is the program that Table 5 executes.

Formalization of compiler state growth, checked lowering, and program compilation: `src/obseq3/compile.lean`.
Flattening and its semantic preservation are in `src/obseq3/proof/spine.lean`. The listing of Fig. 4 is the witness `g14_paper_running_example` in `src/obseq3/compile_tests.lean`.

2.5 Correctness

Correctness is a forward simulation of successful executions. Literal state equality is impossible: the target has registers and introduces route tags (Definition 2.1). The relation instead compares source-observable memory, local pointers, and permission stacks at source-statement boundaries. This section defines exactly the machinery needed to state that relation (Definition 2.2 to Definition 2.6), states the simulation theorems (Theorem 2.7 to Corollary 2.9), and reads the running program as a simulation (Table 7). The proofs are mechanized (Section 3) and are not reproduced here.

2.5.1 Renamings and value simulation.

Definition 2.2 (Renamings) *An address renaming is a partial map $\rho_a : \mathbb{N} \rightarrow \text{option}(\mathbb{N})$ and a tag renaming is a partial map $\rho_t : \text{Tag} \rightarrow \text{option}(\text{Tag})$. A renaming ρ' extends ρ , written $\rho \subseteq \rho'$, when ρ' agrees with ρ wherever ρ is defined.*

The tag map is partial because future tags have not yet been created. The address renaming is kept in the statements for generality, but the invariant below forces it to be the identity, and total, for a reason that combines a property of the two machines with a property of the compiler. Each machine allocates with a deterministic bump allocator that never reuses addresses, and both start at the same watermark. The compiler emits exactly one target allocation, of the same size, for each source allocation, and in the same order. Hence the two watermarks stay equal at every statement boundary, and each fresh block receives the same base on both sides:

$$\rho_a(a) = a \quad \text{for every address } a.$$

Totality is what gives a meaning to the degenerate pointer that an integer-to-pointer cast of an unallocated address yields (**RVAL-FROMEXPOSED**, Table 10). Tags genuinely require renaming: route borrows advance the target tag counter, so a later source and target ref may mint different numeric tags.

Definition 2.3 (Value simulation) *Source cell value v is simulated by target value v' under (ρ_a, ρ_t) , written $v \approx v'$, when one of the following holds.*

- $v = \text{word}(w)$ and $v' = \text{dat}(w)$.
- $v = \text{ptr}(b, o, sz, t)$, $v' = \text{ptr}(b', o, sz, t')$, $\rho_a(b) = b'$, $\rho_t(t) = t'$, and every address in $[b, b + sz)$ is in the domain of ρ_a .
- $v = \text{undef}$, and v' is arbitrary.

Source undef relates to any target value because a successful core source run cannot observe the missing information.

Definition 2.4 (Memory simulation) *Source memory μ_s is simulated by target memory μ_t under (ρ_a, ρ_t) when, for every address a at which μ_s has a cell v , $\rho_a(a)$ is defined and $v \approx \mu_t(\rho_a(a))$.*

The relation is forward only: every present source cell has a related target cell at its mapped address, and extra unobservable target cells are permitted.

2.5.2 Permission simulation.

Definition 2.5 (Permission simulation) *Let items, stacks, and states be those of the Stacked Borrows instance of Section 2.1. Under ρ_t :*

- Two items are related when they have the same constructor and ρ_t maps the source tag to the target tag; raw-pointer items additionally carry the same mutability flag.
- Two stacks are related position by position, so their lengths agree.
- Two stack maps are related when, for every address a , either both lack an entry at a or their entries at a are related stacks.
- Two protector frame lists are related as lists of related tag lists, and two exposed-tag lists are related as tag lists.

Permission states Π_s and Π_t are related, written $\Pi_s \approx \Pi_t$, when their stack maps, protector frames, and exposed lists are related and $\Pi_s.\text{NextTag} \leq \Pi_t.\text{NextTag}$.

Stack-map comparison is by address lookup, not by the list position used to implement the finite map. Disjoint operations may reorder that representation while leaving all address observations the same.

The counter condition is an inequality rather than an equality. Suppose a route borrow mints route tag u at the target counter. Inside the compiled fragment the target stacks contain an extra

item, so the boundary relation is not expected to hold. The store uses u and die removes its item. The target counter remains advanced, which is why a later pair of corresponding source and target tags may have different numeric values.

When a source-visible reference is formed, both machines mint a tag. If they produce t_s and t_t , the proof extends

$$\rho'_t = \rho_{t[t_s \mapsto t_t]}.$$

Because both fresh tags are minted at their machine's counter, and the boundary invariant below keeps every mapped tag strictly below both counters, neither fresh tag is already in the old map's domain or range, so injectivity is preserved. By contrast, a route borrow never extends ρ_t , since there is no source tag to use as a key.

2.5.3 The boundary invariant.

Definition 2.6 (Boundary invariant) *Let C_0 be a compiler state and P a source program. Source state $S = (i, E, \mu_s, \Pi_s)$ and target state $T = (j, R, \mu_t, \Pi_t)$ satisfy the boundary invariant under (ρ_a, ρ_t) , written $S \approx_{C_0, P}^{\rho_a, \rho_t} T$, when there is a compiler state C_i such that the following ten clauses hold.*

- (I1) Control alignment. *Compiling the prefix $P[0..i)$ from C_0 succeeds with state C_i , and $j = C_i.n_i$.*
- (I2) Bound locals. *If $E(\ell) = (a, t)$ for $\ell : \tau$, then $L_i(\ell) = (r, \tau)$ and $R(r) = (\text{PTy}, [\text{ptr}(a, 0, |\tau|, t')])$ with $\rho_a(a) = a$ and $\rho_t(t) = t'$; t is not the wildcard tag; and every address of $[a, a + |\tau|)$ is in the domain of ρ_a .*
- (I3) Source memory. μ_s is simulated by μ_t (Definition 2.4).
- (I4) Permissions. $\Pi_s \approx \Pi_t$ (Definition 2.5).
- (I5) Address identity. ρ_a is the identity wherever defined.
- (I6) Tag-map shape. ρ_t is injective and maps the wildcard tag to itself.
- (I7) Tag bounds. *Every source tag in the domain of ρ_t lies below $\Pi_s.\text{NextTag}$, and every tag in its range lies below $\Pi_t.\text{NextTag}$.*
- (I8) Allocator lockstep. *The source and target bump-allocation watermarks are equal, the two allocation tables are equal, and ρ_a is defined at every address.*
- (I9) Unbound locals. *If E has no binding for ℓ , then L_i has no entry for ℓ .*
- (I10) Register freshness. *Every register in the range of L_i is below $C_i.n_r$.*

The invariant is required only at source-statement boundaries and is deliberately strict about stack shape there. The compiled fragment may push a route tag's item inside a fragment, but die removes it before the invariant is re-established. Numeric next-tag counters may still differ, which is why tags are renamed and the target counter need only be greater.

Each clause beyond (I3) and (I4) is there because a statement of the theorem without it is false or unprovable for some compiled fragment: memory agreement alone cannot show that a fresh load register does not overwrite a local's register (I10), and equal watermarks alone cannot show that an unbound source local corresponds to a target fragment containing `alloc` (I9).

2.5.4 Simulation theorems.

Let `srcStep` fetch the source statement at the current PC and execute it, treating `halt` and a missing statement as successful fixed points. The *core fragment* consists of `halt`, the two protector-frame statements, and assignments, plain or guarded, whose expression is *any* expression of the full language (Appendix A): a constant, a copy, a reference with any retag kind k , protector flag c , and mask m , an uninitialized value, a pointer cast or offset, a slice retag, or either exposed-provenance cast. Here “any kind” includes shared, mutable, both raw pointer kinds, and two-phase.

Only heap alloc and dealloc lie outside the fragment. A program is *core* when every statement in it is.

Theorem 2.7 (One-step forward simulation) *Let P be a core program, C_0 any compiler state, and Q the result of compiling P from C_0 . If*

$$S \approx_{\rho_a, \rho_t}^{C_0, P} T \quad \text{and} \quad \text{srcStep}(P, S) = \text{ok}(S'),$$

then there exist $\rho'_a \supseteq \rho_a$, $\rho'_t \supseteq \rho_t$, a target state T' , and n_t such that

$$Q \vdash T \xrightarrow{n_t} T' \quad \text{and} \quad S' \approx_{\rho'_a, \rho'_t}^{C_0, P} T'.$$

The compiler state in the final invariant is still C_0 . Compilation is not executing alongside the target. The invariant internally recompiles the appropriate prefix to identify the next target label. Only the renamings grow during simulation. The target step count n_t is existential because fragments have different lengths. For `halt` or a missing source statement, $n_t = 0$.

Theorem 2.8 (Forward simulation of finite runs) *Let P be a core program, C_{init} the compiler state with empty code, counters zero, and an empty local map, and Q the result of compiling P from C_{init} . If*

$$S \approx_{\rho_a, \rho_t}^{C_{\text{init}}, P} T \quad \text{and} \quad \text{runS}^{n_s}(P, S) = \text{ok}(S'),$$

then there exist $\rho'_a \supseteq \rho_a$, $\rho'_t \supseteq \rho_t$, a target state T' , and n_t such that

$$\text{runT}^{n_t}(Q, T) = \text{ok}(T') \quad \text{and} \quad S' \approx_{\rho'_a, \rho'_t}^{C_{\text{init}}, P} T'.$$

Theorem 2.8 takes the initial invariant as a premise; it does not hide initialization inside compilation. For closed programs the premise holds outright. Let S_{init} and T_{init} be the source and target states with empty environment, register file, and memory, PC zero, both bump allocators at watermark zero, and both permission states initialized. They satisfy the boundary invariant for every program P , under the identity ρ_a and the ρ_t that maps only the wildcard tag to itself.

Corollary 2.9 (Closed programs) *Let P be a core program and Q its compilation from C_{init} . If $\text{runS}^{n_s}(P, S_{\text{init}}) = \text{ok}(S')$, then there exist ρ'_a , ρ'_t , T' , and n_t with $\text{runT}^{n_t}(Q, T_{\text{init}}) = \text{ok}(T')$ and $S' \approx_{\rho'_a, \rho'_t}^{C_{\text{init}}, P} T'$.*

These results are preservation of successful finite executions. They are not backward simulation, divergence preservation, or an equivalence between source and target error strings. Their observable consequences are the memory and permission clauses of the final invariant, made explicit in Corollary 2.11.

2.5.5 *The running program as a simulation.*

Table 7. The running program as a simulation: the boundary invariant at each source-statement boundary. i is the source PC and $j = C_i.n_l$ the target label (Fig. 4); the ρ_t column shows what the step *adds* (ρ_a is the identity throughout); “next” is the pair of tag counters.

i	j	added to ρ_t	next	what the invariant records at this boundary
0	0	$0 \mapsto 0$	1, 1	Initial states: nothing is bound, and ρ_t fixes only the wildcard.
1	2	$1 \mapsto 1$	2, 2	x is bound on both sides, (I2): equal watermarks (I8) gave both allocations base 0, and the two owning tags form the pair (1, 1).
2	5	—	2, 3	The stacks at cell 1 are again related (I4): borrow; storec; die left the stack effect of the source’s one useMut. Route tag 2 is in no renaming; the counters part.
3	8	$2 \mapsto 3$, $3 \mapsto 4$	4, 5	y is bound, and the reference it holds is related (I3): ρ_t grew by two pairs that are <i>not</i> numerically equal, which (I6) and (I7) permit.
4	10	—	4, 5	Cell 1 holds related words (I3): the loaded ptr(0, 1, 3, 3) and ptr(0, 1, 3, 4) were related, so both writes went through related tags to the same cell.

Example 2.10 Table 7 replays Table 3 against Table 5. Each row is an instance of the invariant that Corollary 2.9 asserts; the first is the initial relation and each later one follows from the previous by Theorem 2.7.

Boundary 1. Statement 0 assigns to the unbound local x . Source preparation allocates $|\tau_x| = 3$ cells and obtains the owning tag 1; the target fragment begins with $R_0 := \text{alloc}_{\rho_x}$, allocates the same base by (I8), and obtains tag 1. The proof extends ρ_t by $1 \mapsto 1$ and the local map by $x \mapsto (R_0, \tau_x)$; the store establishes (I3) for cell 0.

Boundary 2. Statement 1 is $x.1.0 := \text{const}(42)$. The source resolves the place to address 1 and performs $\text{useMut}(\Pi_s, 1, 1, 1)$. The target performs

$$\text{ref}(\Pi_t, 1, 1, 1, \text{mutable}, \text{false}, []) = (\Pi_1, 2), \quad \text{useMut}(\Pi_1, 1, 1, 2) = \Pi_2, \quad \text{die}(\Pi_2, 1, 1, 2) = \Pi'_t.$$

The third event removes the route tag’s item, so the two stacks at cell 1 are again related (I4) under the *unchanged* ρ_t : tag 2 has no source counterpart and never escapes. Both memories update only cell 1, where source word 42 relates to target $\text{dat}(42)$, so (I3) is restored and cells 0 and 2 are framed. (I1) advances from source PC 1 to 2 and from target label 2 to the next prefix label 5. The target counter is now 3 and the source counter 2, which the inequality of Definition 2.5 absorbs.

Boundary 3. Statement 2 allocates y on both sides, with owning tags 2 and 3, and then forms the reference, with tags 3 and 4. Neither pair is an equality, and neither could be: the route borrow of the previous statement advanced only the target counter. This is the step at which a tag renaming, rather than tag equality, is forced. The stored pointers $\text{ptr}(0, 1, 3, 3)$ and $\text{ptr}(0, 1, 3, 4)$ are related by Definition 2.3, and the stacks at cell 1, $[\text{MutRef}(3), \text{Own}(1)]$ and $[\text{MutRef}(4), \text{Own}(1)]$, by Definition 2.5.

Boundary 4. Statement 3 writes through y . Both machines read cell 3 through y ’s owning tag, related by $2 \mapsto 3$, and obtain pointers related by (I3). The two writes therefore go through tags related by $3 \mapsto 4$, to the same cell. No tag is minted and none is retired.

Flattening is semantically load-bearing at boundary 2. A route borrow of the intermediate width two would operate on cell 2 as well, so the target would not simulate the source’s one-cell write in the presence of a borrow of the sibling $x.1.1$. The compiled path must use the composed offset and the final width.

The four formal views of the program thus line up. MIRLite’s typed path has input layout τ_x , selected layout Nat, composed offset one, and width one. OSEA-IR begins with an erased PTy pointer in R_0 , which the compiler’s local map connects to $x : \tau_x$. Flattening prevents an intermediate two-cell borrow, so place lowering creates exactly one route tag for cell 1, and cleanup supplies the matching one-cell die. At each boundary, address identity relates the updated cell, memory simulation frames its neighbours, permission simulation cancels the route tag without adding it to ρ_t , and prefix compilation places the target at the next fragment exactly when the source reaches the next statement:

typed source range $\rightarrow$ flattened target route $\rightarrow$ explicit events $\rightarrow$ boundary simulation.

2.5.6 Observable consequences.

Corollary 2.11 (Observations at a boundary) *If $S \approx_{\rho_a, \rho_t}^{C_0, P} T$, then:*

- if $\mu_s(a) = \text{word}(w)$, then $\mu_t(a) = \text{dat}(w)$;
- if $\mu_s(a) = \text{ptr}(b, o, sz, t)$, then $\mu_t(a) = \text{ptr}(b, o, sz, t')$ with $\rho_t(t) = t'$ and the referent range $[b, b + sz)$ mapped;
- if $E(\ell) = (a, t)$ for $\ell : \tau$, then the register $L_i(\ell)$ holds PTy and exactly one pointer with base a , offset zero, size $|\tau|$, and tag $\rho_t(t)$;
- the target stack at every address has the same length and item constructors as the source stack, with every source tag renamed by ρ_t ;
- the target PC is the next label after compiling exactly $P[0..i)$.

Several asymmetries are intentional. The target may contain extra registers, because value temporaries remain after use. Source undef refines any target cell, so the memory relation has no reverse-domain guarantee. Tag counters need not be equal, and numeric tag equality is not observable. Finally, the invariant is required only at source-statement boundaries; intermediate target states may contain live route tags.

These choices are the minimum abstraction needed by the actual generated code. Strengthening to literal register equality would reject harmless temporaries, while weakening permission stacks to ignore arbitrary target items would no longer justify later accesses.

3 Mechanization and scope

PRECISE CLAIM The mechanized theorems are forward simulations for successful finite runs of core programs: halt, the protector-frame statements, and plain or guarded assignments from every expression of the language. The executable compiler also lowers heap alloc and dealloc; those two statements are outside Theorem 3.7 until their simulation cases are added (Appendix A.5).

Every numbered definition and result of Section 2.5 is a Lean 4 declaration in `src/obseq3/proof/`:

Table 8. The results of Section 2.5 and their Lean declarations.

This paper	Lean declaration	File
Definition 3.3	MemValSim	common.lean
Definition 3.4	SourceMemSim	common.lean
Definition 3.5	PermSim	common.lean
Definition 3.6	CompilerInv	common.lean
core fragment	CoreRhs, CoreStmt, CoreProg	common.lean
Theorem 3.7	CompilerInv_step	compiler.lean
Theorem 3.8	compile_correct	compiler.lean
initial relation	CompilerInv_initial	compiler.lean
Corollary 3.9	compile_correct_from_initial	compiler.lean

The proof is about 17,700 lines and 426 lemmas across the 13 files of that directory. None of it contains an admitted goal. A checked audit prints the axioms that Theorem 3.8 and Corollary 3.9 depend on and fails if that set differs in either direction from a pinned whitelist. The whitelist contains exactly the three standard Lean axioms, propositional extensionality, choice, and quotient soundness, and no `sorryAx`. The last admitted case, the reference-assignment leaves, was closed on 2026-08-31.

The executable compiler is additionally validated by testing: a compiler witness corpus, a ULLBC conformance corpus checked against Miri verdicts, and a differential run that compiles each conformance program and requires the same verdict from both machines. The running program of this paper is part of the witness corpus, both as a golden listing (Fig. 4) and as a differential test; the states of Table 3 and Table 5 are printed by the mechanized interpreters. Tests cover `alloc` and `dealloc` too, but only a discharged simulation case moves a construct inside Theorem 3.7.

Invariant and simulation vocabulary: `src/obseq3/proof/common.lean`. Statement and whole-program forward simulations: `src/obseq3/proof/compiler.lean`. Witness corpus: `src/obseq3/compile_tests.lean`; state dump of the running program: `notes/2026-09-18-paper-running-example.lean`.

A The full executable surface

The main text presents constants, copies, references, assignment, and `halt`. This appendix extends each of its figures and tables, in the same format, to the language the compiler and the theorem actually cover.

A.1 Syntax

Layout rules are as in Section 2.2: `uninit` has any layout; `exposeAddr(p)` has layout `Nat` for a pointer place p and `fromExposed(p)` a pointer layout for a `Nat` place; `ptrCast`, `ptrOffset`, and `refSlice` map pointer places to pointer layouts; the discriminant of a guarded assignment is a `Nat` place. The source semantics of these forms follows the pattern of Table 2: each expression resolves its place, performs the permission events listed for its target counterpart in Table 10 in the same order, and produces cells. A guarded assignment first prepares the root of its destination, *on both paths*, then reads its discriminant exactly as `copy` does, a real read access, and performs the assignment when the value read equals w .

791 MIRLITE

OSEA-IR

792	Expr $\ni e ::= \dots$	Rhs $\ni h ::= \dots$
793	<code>uninit</code>	<code>allocN_{θ}(n)</code>
794	<code>exposeAddr(p)</code>	<code>allocDyn_{θ}(r)</code>
795	<code>fromExposed(p)</code>	<code>expose(r)</code>
796	<code>ptrCast(p)</code>	<code>fromExposed(r)</code>
797	<code>ptrOffset(p, d)</code>	<code>offset(r, d)</code>
798	<code>refSlice(k, c, p)</code>	<code>borrow(k, c, m, $\perp$, r, δ)</code>
799	Stmt $\ni s ::= \dots$	Instr $\ni I ::= \dots$
800	<code>assignIf(p = w, d := e)</code>	<code>memcpy_{θ}(r_d, r_s)</code>
801	<code>alloc(d, len)</code>	<code>dealloc(r)</code>
802	<code>dealloc(p)</code>	<code>skipIf(r, w, n)</code>
803	<code>pushProtectors</code>	<code>pushProt</code> <code>popProt</code>
804	<code>popProtectors</code>	
805	len $::= \text{const}(n) \mid \text{from}(p)$	

806 Fig. 5. The remaining syntax of MIRLite (left) and OSEA-IR (right), extending Fig. 2 and Fig. 3. d in offset
 807 is an integer; a borrow of length $\perp$ retags the rest of the allocation.

808 A.2 The permission model

809 Table 9. The remaining operations of the permission interface of Section 2.1.

810	Operation	Meaning
811	<code>dealloc(Π, a, n, t) = Π'</code>	Deallocate the n cells from a through t and forget their permissions. The item carrying t must grant writes at every cell, and no item of those stacks may be protected.
812	<code>expose(Π, t) = Π'</code>	Record t as exposed, so that a later wildcard access may resolve to it.
813	<code>pushProt(Π) = Π', popProt(Π) = Π'</code>	Open a protector frame; close the innermost frame, ending the protection of every tag registered in it.

817 Tags are drawn from a countable set with a distinguished *wildcard* tag 0, which stands for an
 818 unknown provenance recovered from an exposed integer. Every fresh tag is minted from NextTag
 819 upward, so it is distinct from all earlier ones and a tag minted later is numerically larger. The rules
 820 of Table 1 extend as follows. *Protectors*: if the flag c of a `ref` is set, the fresh tag is registered in
 821 the innermost protector frame; **SB-READ**, **SB-USE-MUT**, and **SB-DIE** fail when an item they would
 822 disable or remove is protected. *Masks*: entry i of the mask m says whether cell $a + i$ of the retagged
 823 range lies inside an UnsafeCell; a cell beyond the end of m counts as unmarked, so the empty
 824 mask marks nothing. On the cells that m marks, a shared or raw-constant retag performs no access
 825 and inserts RawPtr(true, u) directly above the item carrying t , as **SB-REF-RAW** does. The mask has
 826 one entry per cell, rather than being a single flag, because one reference may cover both kinds
 827 of cell: in `&(i32, Cell<i32>)` the first field is frozen and the second is not, so a single retag
 828 with mask [false, true] pushes Ref(u) on the first cell and inserts RawPtr(true, u) on the second.
 829 *Two-phase*: a two-phase retag performs a read and inserts RawPtr(true, u) likewise, modelling a
 830 reservation that stays writable until activation. *Raw-pointer groups*: **SB-USE-MUT** also accepts $\iota =$
 831 RawPtr(true, t), and then keeps the contiguous run of mutable raw-pointer items directly above ι ,
 832 removing only what lies above the whole group. *Wildcard*: an access through tag 0 resolves to the

topmost exposed item that grants it. This rule is a determinization: for programs that use integer-to-pointer casts, the theorem is a statement about it rather than about Miri’s angelic choice.

A.3 OSEA-IR

Table 10. The remaining OSEA-IR rules, extending Table 4. In the rules that read through r , $T.R(r) = (\theta_r, [\text{ptr}(b, o, sz, t)])$, $a = b + o$, and $a < b + sz$. wild is the wildcard tag and $\text{resolve}(\mu_t, n)$ looks n up in the allocation table.

Premises	Conclusion	Rule Name
$sz = n \cdot \text{typeSize}(\theta)$, $\text{alloc}(T.\mu_t, sz) = (b, \mu'_t)$, $\text{own}(T.\Pi_t, b, sz) = (\Pi'_t, u)$	$T \vdash \text{allocN}_\theta(n) \Downarrow_h$ $(\text{PTy}, [\text{ptr}(b, 0, sz, u)], T[\mu_t \mapsto \mu'_t, \Pi_t \mapsto \Pi'_t])$	RVAL-ALLOCN
$\text{read}(T.\Pi_t, a, 1, t) = \Pi_1$, $T.\mu_t(a) = \text{dat}(n)$, $sz = n \cdot \text{typeSize}(\theta)$, $\text{alloc}(T.\mu_t, sz) = (b', \mu'_t)$, $\text{own}(\Pi_1, b', sz) = (\Pi_2, u)$	$T \vdash \text{allocDyn}_\theta(r) \Downarrow_h$ $(\text{PTy}, [\text{ptr}(b', 0, sz, u)], T[\mu_t \mapsto \mu'_t, \Pi_t \mapsto \Pi_2])$	RVAL-ALLOCDYN
$\text{read}(T.\Pi_t, a, 1, t) = \Pi_1$, $T.\mu_t(a) = \text{ptr}(b', o', sz', t')$, $\text{expose}(\Pi_1, t') = \Pi_2$	$T \vdash \text{expose}(r) \Downarrow_h$ $(\text{NatTy}, [\text{dat}(b' + o')], T[\Pi_t \mapsto \Pi_2])$	RVAL-EXPOSE
$\text{read}(T.\Pi_t, a, 1, t) = \Pi_1$, $T.\mu_t(a) = \text{dat}(n)$, $\text{resolve}(T.\mu_t, n) = (b', o', sz')$	$T \vdash \text{fromExposed}(r) \Downarrow_h$ $(\text{PTy}, [\text{ptr}(b', o', sz', \text{wild})], T[\Pi_t \mapsto \Pi_1])$	RVAL-FROMEXPOSED
$\text{read}(T.\Pi_t, a, 1, t) = \Pi_1$, $T.\mu_t(a) = \text{ptr}(b', o', sz', t')$, $o' + d \geq 0$	$T \vdash \text{offset}(r, d) \Downarrow_h$ $(\text{PTy}, [\text{ptr}(b', o' + d, sz', t')], T[\Pi_t \mapsto \Pi_1])$	RVAL-OFFSET
$T.R(r) = (\theta_r, [\text{ptr}(b, o, sz, t)])$, $a' = b + o + \delta$, $\text{ref}(T.\Pi_t, a', sz - (o + \delta), t, k, c, m) = (\Pi'_t, u)$	$T \vdash \text{borrow}(k, c, m, \perp, r, \delta) \Downarrow_h$ $(\text{PTy}, [\text{ptr}(b, o + \delta, sz, u)], T[\Pi_t \mapsto \Pi'_t])$	RVAL-BORROW-REST
$Q(j) = \text{memcpy}_\theta(r_d, r_s)$, $n = \text{typeSize}(\theta)$, $R(r_d) = (\theta_d, [\text{ptr}(b_d, o_d, sz_d, t_d)])$, $a_d = b_d + o_d$, $R(r_s) = (\theta_s, [\text{ptr}(b_s, o_s, sz_s, t_s)])$, $a_s = b_s + o_s$, both ranges fit and do not overlap, $\text{read}(\Pi_t, a_s, n, t_s) = \Pi_1$, $\text{useMut}(\Pi_1, a_d, n, t_d) = \Pi_2$	$(j, R, \mu_t, \Pi_t) \xrightarrow{\text{sea}}$ $(j + 1, R, \mu_t[a_d \mapsto \mu_t[a_s..a_s + n]], \Pi_2)$	EXEC-MEMCPY
$Q(j) = \text{dealloc}(r)$, $R(r) = (\theta_r, [\text{ptr}(b, 0, sz, t)])$, $\text{dealloc}(\Pi_t, b, sz, t) = \Pi'_t$	$(j, R, \mu_t, \Pi_t) \xrightarrow{\text{sea}}$ $(j + 1, R, \mu_t \setminus [b, b + sz], \Pi'_t)$	EXEC-DEALLOC
$Q(j) = \text{skipIf}(r, w, n)$, $R(r) = (\theta_r, [\text{dat}(w')])$, $j' = j + 1$ if $w' = w$, else $j' = j + 1 + n$	$(j, R, \mu_t, \Pi_t) \xrightarrow{\text{sea}} (j', R, \mu_t, \Pi_t)$	EXEC-SKIPIF
$Q(j) = \text{pushProt}$	$(j, R, \mu_t, \Pi_t) \xrightarrow{\text{sea}} (j + 1, R, \mu_t, \text{pushProt}(\Pi_t))$	EXEC-PUSHPROT
$Q(j) = \text{popProt}$, $\text{popProt}(\Pi_t) = \Pi'_t$	$(j, R, \mu_t, \Pi_t) \xrightarrow{\text{sea}} (j + 1, R, \mu_t, \Pi'_t)$	EXEC-POPPROT

The pointer right-hand sides are operational rather than mere casts, and share a three-stage pattern: validate the register shape, perform the permission events in source order, construct a result. They read *through the outer pointer in the register* before inspecting the value in memory. For example, `offset` does not change the offset of that outer pointer: it loads a pointer-valued cell, changes the loaded value, and returns the result in a register. This is why the compiler can use it to implement MIRLite pointer arithmetic without suppressing the source’s read event. `allocDyn` reads its length before it allocates, which matches the source’s permission-visible length read. The bump allocator never reuses a deallocated range. `skipIf` touches neither memory nor permissions: the discriminant it tests was loaded into a register by an ordinary load. The compiler emits no `memcpy`; the instruction remains part of the machine.

OSEA-IR errors fall into four semantic classes rather than one generic “stuck” state:

Table 11. Error classes of OSEA-IR.

Class	Typical cause	Where detected
Register shape	Missing register, non-pointer singleton, or wrong source runtime type.	Before any memory or permission change.
Value shape	A cast or dynamic allocation observes the wrong memory-value constructor.	After any required permission read, so that event order remains explicit.
Spatial	A complete access range exceeds its allocation, a pointer offset becomes negative, or memcpy overlaps.	Before the corresponding read/write permission event.
Permission	read, ref, useMut, die, dealloc, or a protector pop rejects the operation.	At the permission interface; its error is propagated.

The forward simulation only starts from a successful source step. It therefore proves that the compiled fragment avoids all of these errors for the covered cases; it does not claim that source and target failure messages coincide.

A.4 Compiler

Every expression has the pre-code/store split of **RHS-COPY**. In Table 12, “lower p ” is $C \vdash_{\text{shared}} p \Downarrow_{\text{cP}}(r_s, D_s, C_1), r_v$ is a fresh value register, and the pre-code is emitted before the destination is lowered.

Table 12. Lowering of the remaining expressions, extending **RHS-CONST**, **RHS-COPY**, and **RHS-REF** of Table 6. The selected layout is τ ; in ptrOffset, $p : \text{Ptr } \sigma$.

Expression	Pre-destination code	Deferred store $F(r_d)$
uninit	None.	$\text{store}_{\tau}([\text{undef}]^{ \tau }, r_d)$
exposeAddr(p)	Lower $p; r_v := \text{expose}(r_s); \text{cleanup}(D_s)$.	$\text{store}_{\text{NatTy}}(r_v, r_d)$
fromExposed(p)	Lower $p; r_v := \text{fromExposed}(r_s); \text{cleanup}(D_s)$.	$\text{store}_{\text{PTy}}(r_v, r_d)$
ptrCast(p)	Lower $p; r_v := \text{load}_{\text{PTy}}(r_s); \text{cleanup}(D_s)$.	$\text{store}_{\text{PTy}}(r_v, r_d)$
ptrOffset(p, d)	Lower $p; r_v := \text{offset}(r_s, d \cdot \sigma); \text{cleanup}(D_s)$.	$\text{store}_{\text{PTy}}(r_v, r_d)$
refSlice(k, c, p)	Lower $p; r_v := \text{load}_{\text{PTy}}(r_s); \text{cleanup}(D_s)$; then $r_v := \text{borrow}(k, c, [], \perp, r_v, 0)$.	$\text{store}_{\text{PTy}}(r_v, r_d)$

$[\text{undef}]^{| \tau |}$ is a list of exactly $| \tau |$ undefined cells. Pointer offsets are expressed by the source in pointee units, so the compiler scales d by the pointee width before OSEA-IR sees it. A pointer cast is tag-preserving: a borrow would mint a new tag, whereas a load and a store copy the pointer value unchanged while performing the source’s read and write. The placement of cleanup is semantic. In refSlice the mint comes *after* the cleanup: a mutable retag through the loaded tag would pop a route tag still on the stack, and a shared one would bury it, so the route’s die would no longer find its item on top. Every bracket the compiler opens therefore closes with its route tag on top.

Table 13. Lowering of the remaining statements, extending **STMT-ASSIGN** and **STMT-HALT** of Table 6.

Statement	Emitted sequence
<code>alloc(d, const(n))</code>	$\text{root}(d)$; lower d mutable to (r_d, D_d) ; fresh $r_h := \text{allocN}_{[t]}(n)$; $\text{store}_{\text{PTy}}(r_h, r_d)$; $\text{cleanup}(D_d)$.
<code>alloc(d, from(p))</code>	As above, with the heap pointer obtained by: lower p shared to (r_p, D_p) ; $r_h := \text{allocDyn}_{[t]}(r_p)$; $\text{cleanup}(D_p)$.
<code>dealloc(p)</code>	Lower p shared to (r_p, D_p) ; fresh $r_v := \text{load}_{\text{PTy}}(r_p)$; $\text{cleanup}(D_p)$; $\text{dealloc}(r_v)$.
<code>assignIf(p = w, d := e)</code>	$\text{root}(d)$, before the guard; lower p shared to (r_p, D_p) ; fresh $r_g := \text{load}_{\text{NatTy}}(r_p)$; $\text{cleanup}(D_p)$; reserve the label j_g ; compile $d := e$ by STMT-ASSIGN patch $K(j_g) = \text{skipIf}(r_g, w, n)$ with n the number of labels the assignment emitted.
<code>pushProtectors</code>	<code>pushProt</code>
<code>popProtectors</code>	<code>popProt</code>

For `alloc`, destination resolution precedes the evaluation of a dynamic length, matching the source allocation statement rather than the ordinary assignment rule. For `assignIf`, the root of the destination is allocated before the guard because a root allocated inside the guarded fragment would be recorded in L at compile time but would exist at run time only when the guard is taken. The guard *reserves* its label and is patched once the body has been compiled, so the body is compiled exactly once and its measured length is the skip count; no statement is compiled twice.

A.5 What remains outside the theorem

Two statements are executable and tested but not yet inside Theorem 1.7. Adding them is not “supporting the constructor”; it is preserving the ten clauses of Definition 1.6 through each emitted sequence. For `alloc`, the proof must synchronize a runtime or constant length and extend both renamings over the fresh heap block. For `dealloc`, it must relate the offset-zero checks, the permission retirement, and the removal of the same memory range. Neither follows merely from successful differential tests.